A Project Report on

AI-BASED PERSONALIZED LEARNING RECOMENDATION SYSTEM

1. Problem Understanding:

Many students feel confused about what to study and which resources to use. Every student has different strengths, weaknesses, and learning styles. However, most learning platforms provide the same content to everyone.

For example, a student weak in Mathematics may need a strong foundation course, while a student preparing for JEE may need advanced Physics content. Without proper guidance, students may waste time on unsuitable materials.

To solve this problem, we developed an AI-based system that suggests suitable courses based on student input. The system also collects feedback and improves its recommendations over time.

Our goal was to build a smart and simple learning recommendation platform that gives personalized suggestions instead of general advice.

2. Design and Architecture:

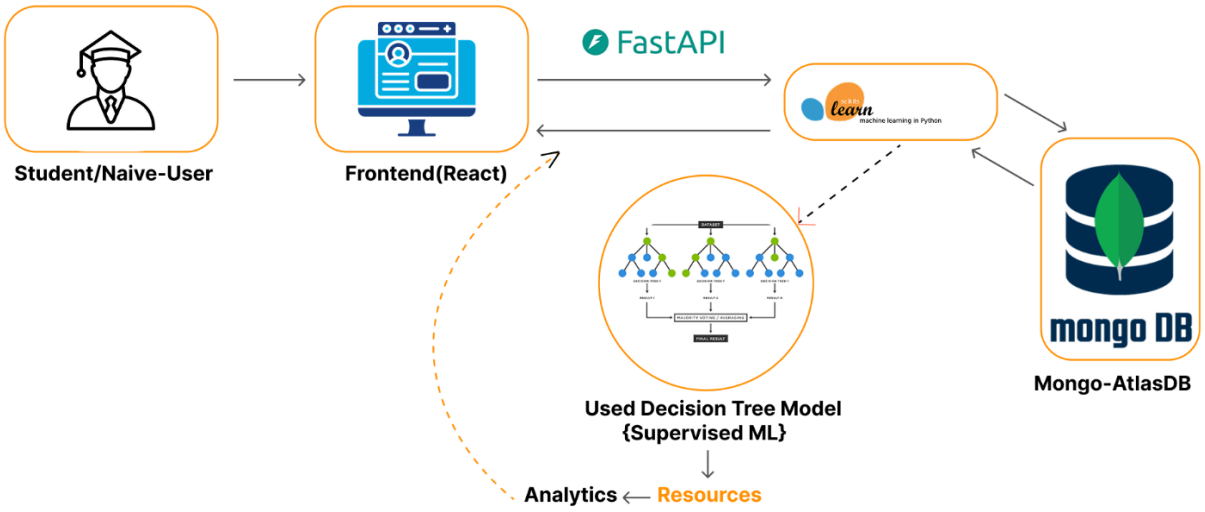


Fig 1. Workflow diagram

Overall Flow:

Student → Frontend → Backend → Machine Learning Model → Database → Result.

Frontend (React):

- Collecting student details (class level, weak subject, learning style)
- Sending data to the backend
- Displaying recommended course and resources
- Collecting feedback
- Showing analytics using charts

Backend (FastAPI - Python):

The backend:

- Receives requests form frontend
- Processes the data
- Sends input to the ML model
- Stores data in MongoDB
- Returns results to frontend

It acts as the main control system of the project.

Machine Learning Model:

We used a random forest classifier for prediction.

- Takes student details as input
- Predicts the most suitable course
- Provides a confidence score

The model is retrained automatically after every 20 feedback entries to improve.

Database (MongoDB):

MongoDB is used to store:

- Student inputs
- Recommended courses
- Confidence scores
- Feedback ratings
- Timestamps

This stored data is also used for analytics and retraining.

3. Implementation Details

Step 1: Input Collection

The student enters:

- Class Level
- Weak Subject
- Learning Style
- Goal

The data is stored using React state management.

Step 2: Sending Data to Backend

The frontend sends data to the backend using HTTP requests (Axios).

Step 3: Data Processing and Prediction

The backend:

- Converts categorical data into numerical format(one-hot-encoding)
- Sends it to the trained Random Forest model
- Gets prediction result
- Calculates confidence score

Step 4: Displaying Results

The frontend displays:

- Recommended course
- Confidence percentage
- YouTube videos
- Book suggestions

Step 5: Feedback System

After receiving recommendation:

- User selects “Helpful” or “NotHelpful”
- Optimal comment can be added
- Feedback is stored on MongoDB

Step 6:Retraining System

After every 20 feedback entries:

- The retraining script runs
- Old data from database is used as training dataset
- Model is retrained
- New model is saved

This makes the system adaptive and self-improving.

Step 7: Analytics Dashboard

The analytics section shows:

- Total recommendations
- Helpful percentage
- Not helpful percentage
- Most recommended course
- Average confidence
- Course distribution (bar chart)

This helps us analyze system performance.

4. Results and Test Cases:

We tested the system using multiple student inputs.

Test case 1:

Input:

- Class Level:10
- Weak Subject: Math
- Learning Style: Visual

Output:

- Recommended Course: Math Foundation
- Confidence: High

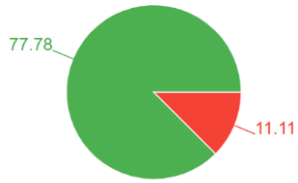
Test Case 2:

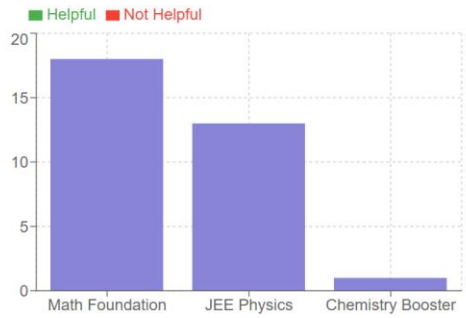
Input:

- Class Level:11
- Weak Subject: Physics
- Goal :JEE

Output:

- Recommended Course: JEE Physics

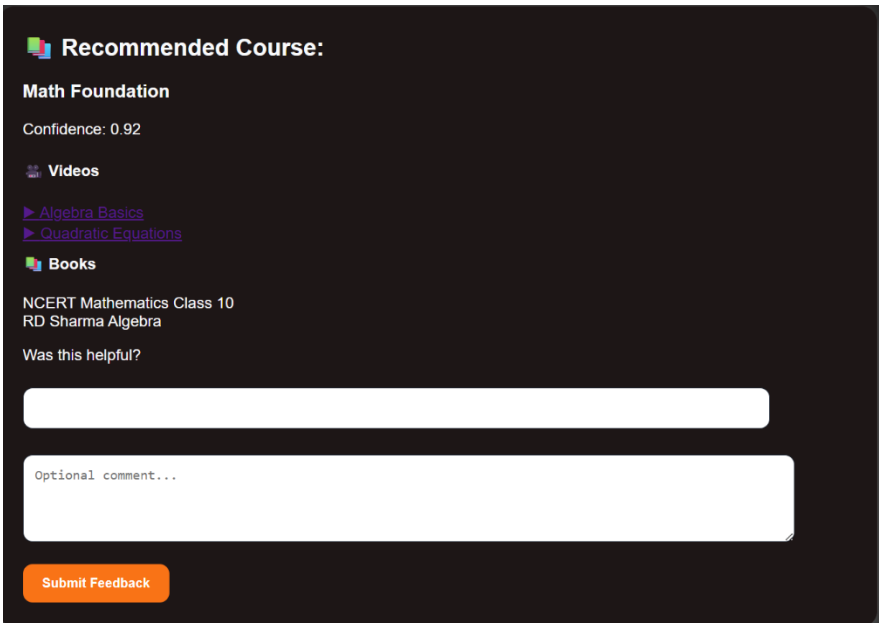




Total Recommendations: 32

Total Feedback: 9

Top Course: Math Foundation



Feedback Testing

- Feedback was successfully stored in MongoDB.
- After 20 feedback submissions, the retraining process was triggered.
- Analytics updated correctly.

5. Reflections and Future Scope:

Through this project we learned,

- Frontend and backend communication using APIs
- Integration of machine learning into a real application
- Storing and managing data using MongoDB
- Retraining a model using real-time feedback
- Visualize data using charts

6. Future scope

In future we can improve this system by,

- Adding user login and authentication using JWT
- Using advanced ML models for better accuracy
- Deploying the project on cloud platform
- Creating a mobile application
- Adding more courses and subjects

7. Team Member Contribution

Team member	Technical work	Report/Presentation
A.Harshavardhan Reddy- [24881A05Y7]	Developed FastAPI backend, implemented machine learning model using scikit-learn, integrated MongoDB, and built the auto-retraining pipeline. Technical content and architecture.	Contributed to project - Video presentation.
Y.VinayChary- [24881A05BY]	Designed and developed React frontend, implemented UI/UX components, styled dark theme interface, and handled user interaction flows	Contributed to report Documentation.